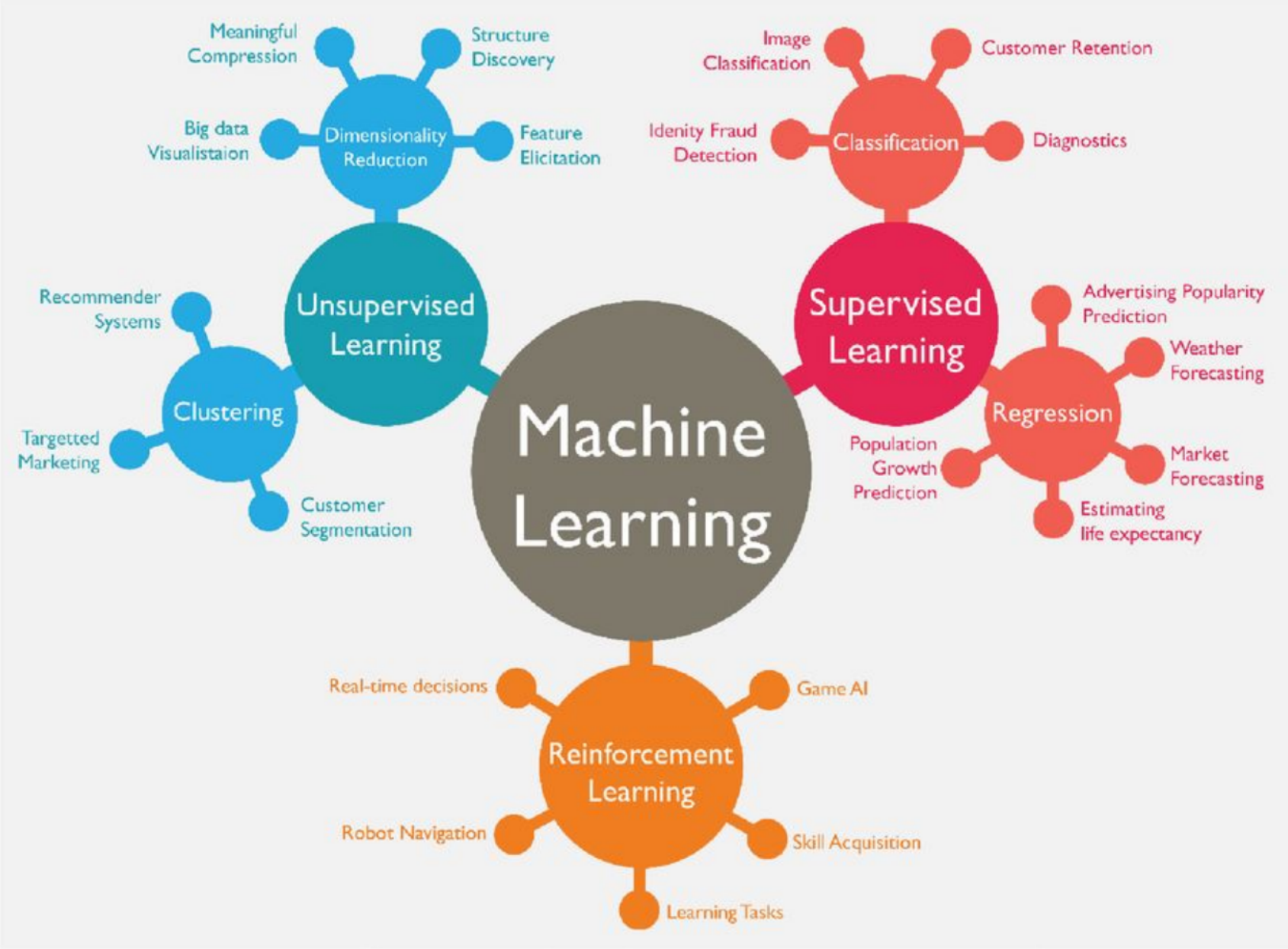
TP OST – Décision dans l'incertain & Sécurité

Introduction aux algorithmes de bandit

Radu Ciucanu

radu.ciucanu@insa-cvl.fr

Reinforcement learning



Source: https://www.mactores.com/services/aws-big-data-machine-learning-cognitive-services/

Stochastic Multi-Armed Bandits
- Motivation
- Formalization
- UCB algorithms
- Linear bandits
- Contextual bandits
- Hierarchical bandits

My research

These slides are based on slides by Marta Soare.

Why should we care about bandits?

Decision making under uncertainty is a significant, widely encountered challenge.

Bandits capture key aspects of this challenge, summarized in the **exploration-exploitation trade-off**.

In machine learning, this framework is now very popular, used in practice by big companies, and growing fast.

Why should we care about bandits?

Some examples of the exploration-exploitation trade-off

- Which road should I take to work, based on the previously observed delays at this time?
- Where to have dinner? Places I know or a new restaurant?
- How to divide my study time between courses?
- Which version of a website would generate the most revenue?
- Which ad will obtain more clicks?
- What move should be considered next when playing go/chess?

Name origin (1/2) : Clinical trials

Bandit problems were introduced by Thompson, in a paper from 1933.

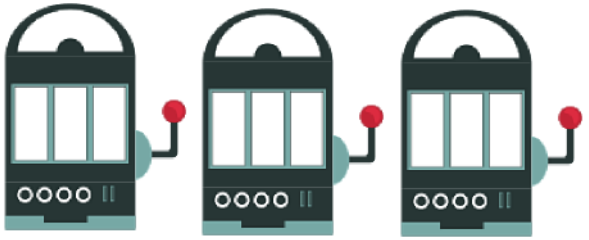
Thompson was interested in medical trials and the inefficiency of fixing the treatment allocation beforehand, without adapting as drugs appear to be more or less effective.

Name origin (1/2) : Clinical trials

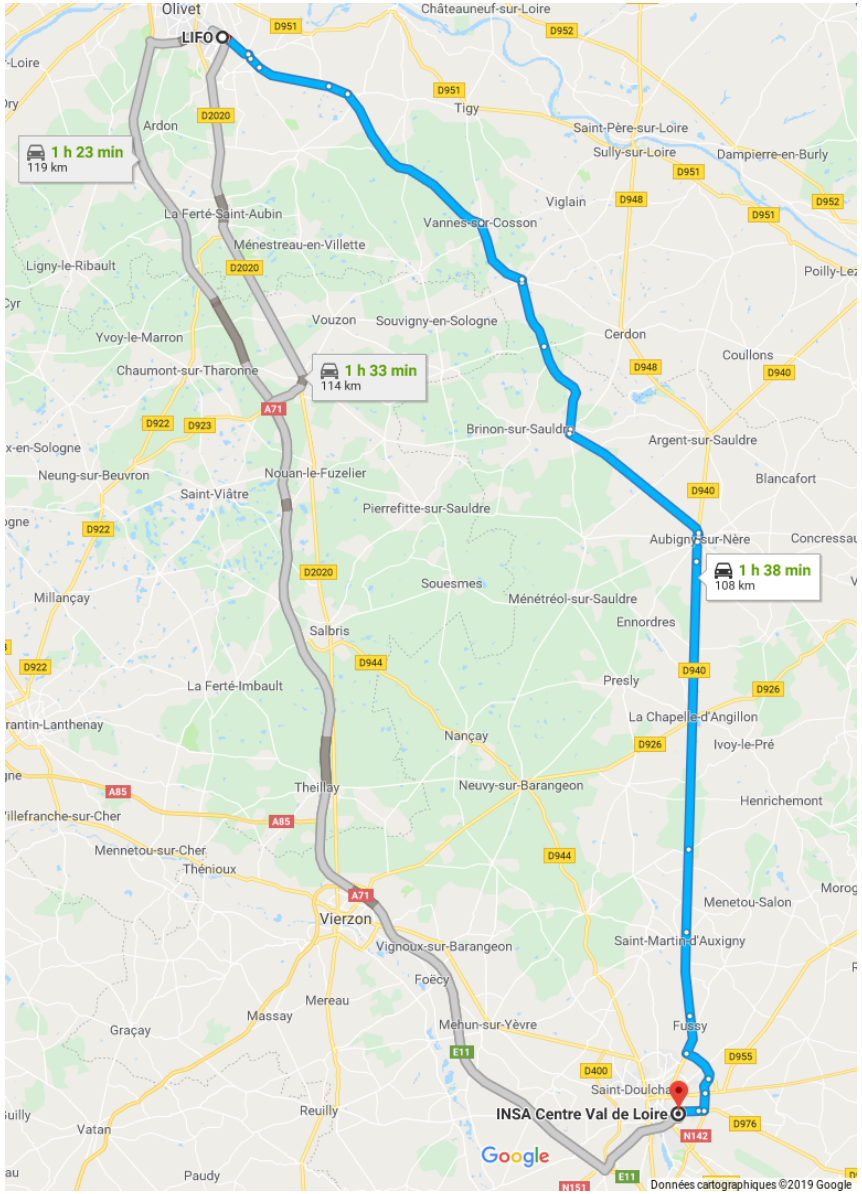
- As introduced in [Thompson, 1933] :
- K drugs with unknown effect.
- Assume the drugs have the same effects on all patients.
- Choose one drug at a time and observe its effect.
- **How to choose the best drug given the effects observed on previous patients?**

Name origin (2/2) : Slot machines

- Studies in the 1950' about animal learning: mice in T-maze.
- Generalized to humans: two-armed slot-machines (bandits).
- Analogy to casinos:
  - A player has a finite gambling *budget* (n coins)
  - In the casino there are K slot machines (one-armed bandits), unknown returns.
  - Sequentially pull one arm and observe a reward specific to the chosen arm.



- Goal: **use the n coins to get as much reward as possible.**



- **Question:** which route should we take?
- **Difficulty:** each time we travel, we obtain a limited feedback: traveling time and conditions for the chosen route.
- We cannot learn without repeatedly trying different options.
- **Approach:** trade off between exploration and exploitation.

Exploration vs. Exploitation trade-off

At each time step, the learner can decide either

- **to explore** = gain more information about the machines (some potentially not explored yet)
- **to exploit** = keep pulling the machines that seem to have the highest rewards.

2-armed bandit example

Payoffs after 5 pulls of each arm:
Left arm: 0, 10, 0, 0, 10
Right arm: 10, 0, 0, 0, 0
Budget: 20 more trials (pulls). How many times to pull each arm?

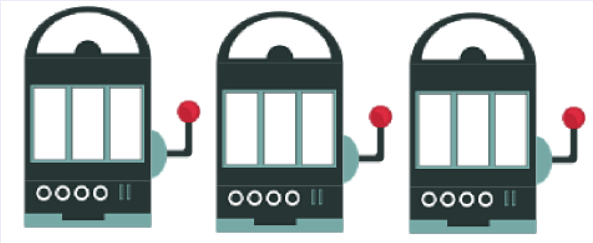
Left arm average payoff: 4 dollars per round;
Right arm average payoff: 2 dollars per round.

Shall we pull left only (**exploit**)?
Shall we pull also right (**explore**)?

The stochastic MAB game: Interaction protocol

The stochastic multi-armed bandits game

The player knows: K – number of arms
N – total budget (number of allowed pulls)
Uncertain environment: $x_1, \ldots, x_K$ – unknown reward distributions
For $t = 1, \ldots, N$
Player selects arm i
Player observes a reward $r_t \sim x_i$
Goal (depending on the application):
find best arm or maximize $\sum_{t=1}^{N} r_t$



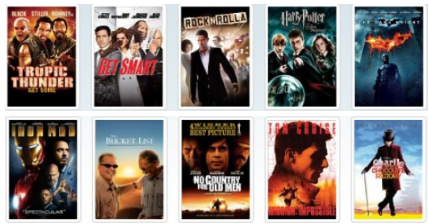
- When deciding which arm to select at time-step t, the player uses the $t-1$ previously observed rewards.
- The player has to decide whether to
  - **exploit**: pull the arm that seems to have the best reward
  - **explore**: get information about the most uncertain arms.

Multi-Armed Bandits - Application Examples

Clinical trials: which treatment to choose for a new patient?
Reward: observed effect on the patient
Exploit: give the treatment that had the best effects
Explore: try a treatment not yet/enough tested

On-line ads
Reward: the user clicks/does not click on the add
Exploit: show the add that received most clicks
Explore: try a new add

Recommendation system
Reward: score given for the recommended product
Exploit: recommend the most popular product
Explore: recommend new/less known products



Performance Measure: Regret Definition (1/2)

- Let $\mu_i = \mathbb{E}[x_i]$ be the expected value of arm i (for $i = 1, \ldots, K$)
- Let $\mu^* = \max_{i=1,\ldots,K} \mathbb{E}[x_i]$ be the best expected value and denote the best arm by x^* ($\mathbb{E}[x^*] = \mu^*$)
- The cumulative **regret** after n pulls:

$$R_n = \mathbb{E}\left[n\mu^* - \sum_{t=1}^{n} r_t\right]$$

  and can be rewritten as:

$$R_n = \sum_{i=1}^{K} (\mu^* - \mu_i)\mathbb{E}[N_i],$$

  where N_i is the number of pulls of arm x_i.
- Goal: **maximize the sum of rewards** $\Leftrightarrow$ **minimize regret**.

Performance Measure: Regret Definition (2/2)

Recall the regret definition $R_n = \sum_{i=1}^{K}(\mu^* - \mu_i)\mathbb{E}[N_i]$.

The regret of our strategy

Can be thought of as **the difference between**
(i) the **sum of rewards of an oracle** strategy which knows the best arm beforehand and chooses it in all n steps and
(ii) the **sum of rewards obtained by our strategy** which needs to discover the values of the arms.

How to balance exploration/exploitation such that the regret is sub-linear?

How would you allocate the budget?

Some strategies

- **Explore:** Draw each arm N/K times.
- **Exploit:** Always pull the empirical best arm $x_{t+1} = \arg\max_i \hat{\mu}_{i,t}$.

- ϵ-**Greedy:** Pull the empirical best arm with probability $1 - \epsilon$ or a random arm with probability ϵ. *Also known as* **Follow-the-Leader**.
- **Forced exploration phase:** begin with pulling uniformly all the arms, then:
  - pull the "best arm" until the end of the budget *Also known as* **Explore-then-Commit**.
  - divide the remaining pulls proportionally to the observed average rewards, etc

---

- **Probability matching:** or Bayesian exploration - assign priors to the arm distributions and select according to the posterior distributions.
- **Optimism in the face of uncertainty:** prefer actions with uncertain values $\rightarrow$ behave as if **the highest statistically plausible value for an arm is indeed its true value**.

Stochastic Multi-Armed Bandits
- Motivation
- Formalization
- UCB algorithms
- Linear bandits
- Contextual bandits
- Hierarchical bandits

My research

Optimistic strategies

First introduced in [Robbins, 1952]

Key ingredients:

1. Build arm indices *(or arm scores)* that take into account both
   (i) the observed **empirical mean** *exploitation term* and
   (ii) the **uncertainty** in the arm value *exploration term*.

2. Optimistic selection: at each time step choose the arm with the largest index.

Optimistic strategies **encourage exploration** up to the extent of **statistical plausibility**.

UCB-type algorithms

The arm indices are **U**pper **C**onfidence **B**ounds on the values of the arms.

In the `UCB-1` algorithm of [Auer et al., 2002], the **index** $B_{i,t+1}$ for arm x_i is given by

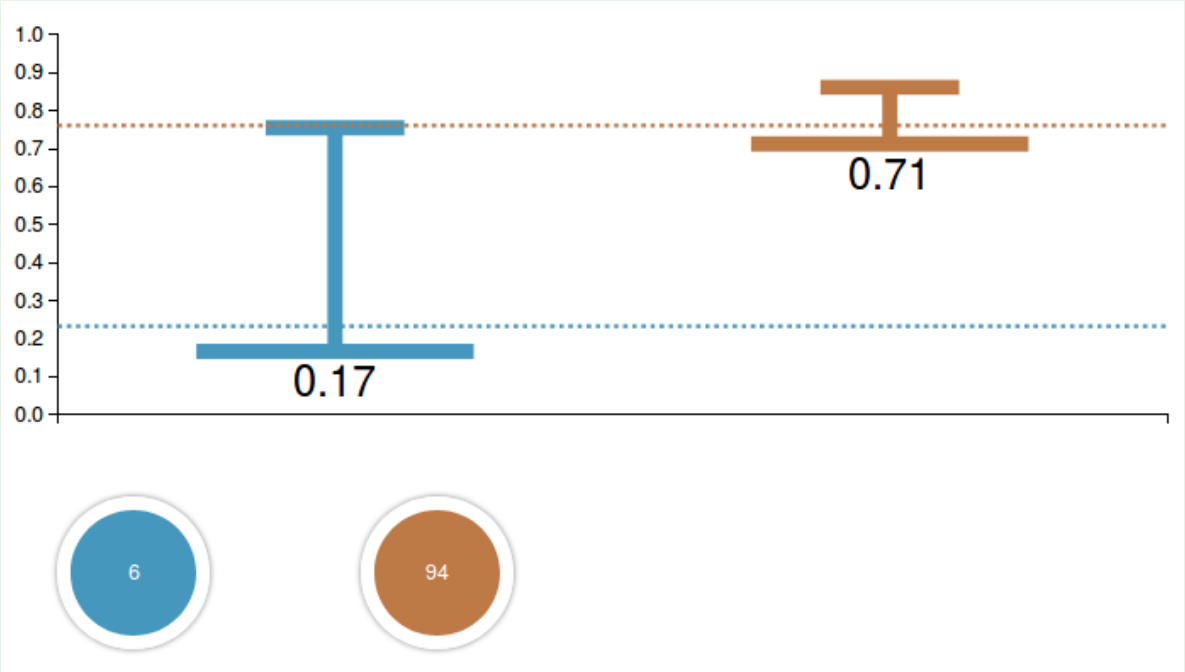
$$B_{i,t+1} = \widehat{\mu}_{i,t} + \sqrt{\frac{2 \ln t}{N_{i,t}}},$$

where t is the number of total rewards observed so far and $N_{i,t}$ is the number of rewards coming from arm x_i.

UCB-type algorithms

The arm indices $B_{i,t}$ are the **largest statistically plausible** value of the arm, given the current available observations.

Example



UCB-1 [Auer et al., Machine Learning Journal 2002]

Input: budget N, K arms

Initialization phase: Pull each arm once and initialize variables

Let $R = 0$ /* *The sum of observed rewards for all arms* */

For $1 \leq i \leq K$

  Let $r = pull(i)$ /* *Random reward for arm i* */

  Let $s_i = r$ /* *Sum of observed rewards for arm i* */

  Let $n_i = 1$ /* *Number of times the arm i has been pulled* */

  Let $\hat{x}_i = r$ /* *Empirical mean for arm i* */

  Let $R = R + r$

  Let $B_i = \hat{x}_i + \sqrt{\frac{2 \times \ln(K)}{n_i}}$

Exploration-Exploitation phase: Pull one selected arm at each time step t

For $K + 1 \leq t \leq N$ /* *Only a budget of $N - K$ is left* */

  Let $a = \arg\max_{1 \leq i \leq K}(B_i)$ /* *a is the arm with maximum B value* */

  Let $r = pull(a)$

  Let $s_a = s_a + r, n_a = n_a + 1, \hat{x}_a = s_a / n_a$

  Let $R = R + r$

  For $1 \leq i \leq K$

    Let $B_i = \hat{x}_i + \sqrt{\frac{2 \times \ln(t)}{n_i}}$

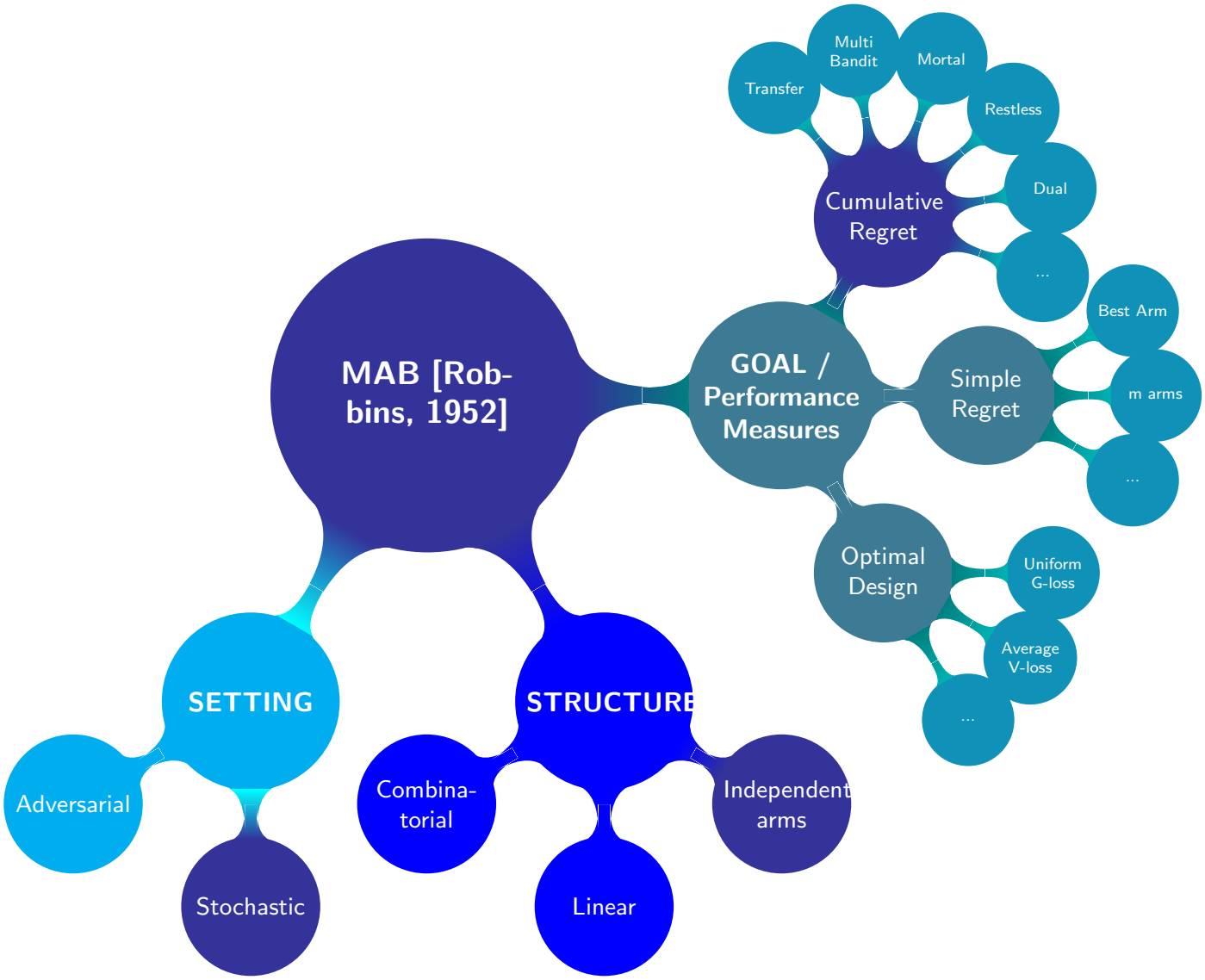
Return R /* *Return sum of observed rewards for all arms* */

Discussion

Several strong assumptions appear in this stochastic multi-armed bandit setting:

- Known (and finite) number of arms
- Stationarity: the set of arms is fixed and the best arm is always the same
- No connection between the arms: What if the number of arm values to be estimated is larger than the budget?
- How to relax some of these assumptions to use UCB-type algorithms in real-life applications?

Some extensions to the stochastic MAB setting



Linear Stochastic Bandits (1/2)

- Presence of a **global structure**: relax the assumption of independence between arms $\rightarrow$ **more realistic applications**.

- **Stochastic Linear** reward function: **successful model**, allows for **geometric intuition**.

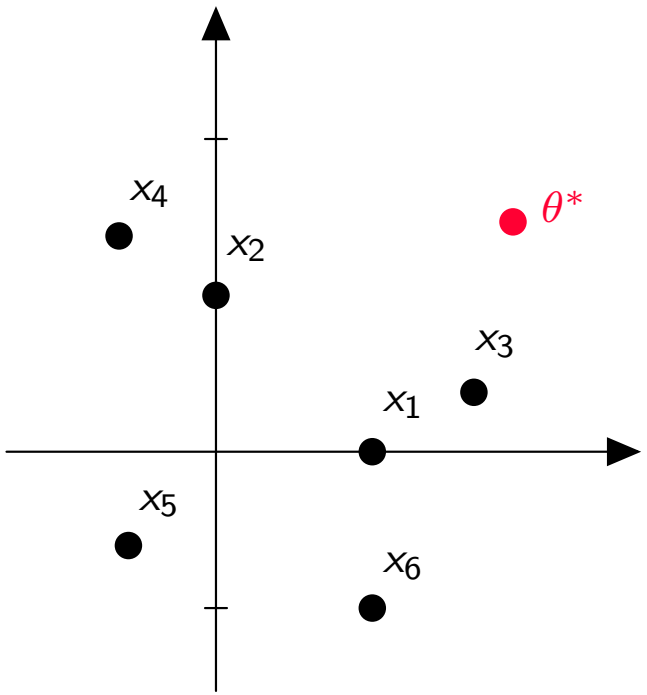
- Many of the problems well studied in MAB have very different solutions in this setting:
  - Introduce **new formalizations** in the literature
  - Require specific and **novel allocation strategies**
  - Tools from online linear optimization.

Linear Stochastic Bandits (2/2)

Finite set of arms:
$\mathcal{X} = \{x_1, x_2, \ldots, x_K\} \subseteq \Re^d$
features describing the items

Assumption:
when sampling arm $x \in \mathcal{X}$
$r(x) = x^\top \theta^* + \eta$
$\theta^* \in \Re^d$ unknown parameter
weights given to the features
η bounded noise.

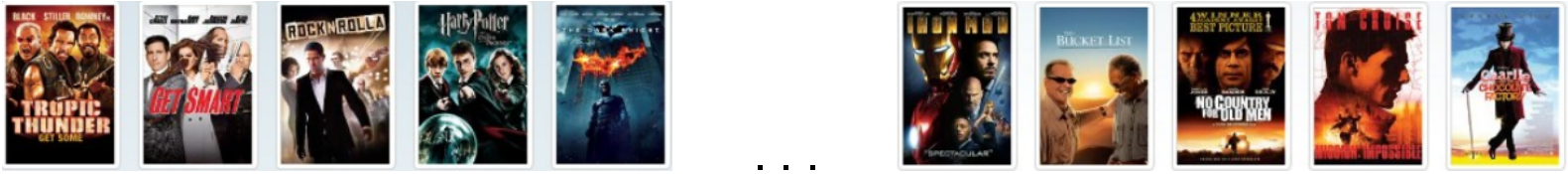
Uncertainty captured in θ^*:
global **structure**
estimate **d** features, instead of K means.



Contextual bandits

Modern motivations: recommender systems, news/ads selection, website optimization, other personalized interactions.
All these depend on a context (user profile, preferences) that needs to be taken into account for regret minimization.

Example: Movie Recommender System



For the t-th visitor of an web-site:

- recommend a movie
- observe a rating for that movie (for instance, 1-5 stars)
- goal: maximize the sum of ratings

The best movie strictly depends on the t-th visitor (best arm changes from context to context). How to take the context into account?

Bandits for tree search

In particular used in planning, with applications in game playing: **What move should be considered next when playing go/chess?**

Choose the move depending on the highest possible reward obtainable if a move is chosen (estimation and optimistic selection). Update estimates sequentially.

Exciting developments recently in the game of Go.

Stochastic Multi-Armed Bandits
Motivation
Formalization
UCB algorithms
Linear bandits
Contextual bandits
Hierarchical bandits

My research

My research: Secure protocols for bandit algorithms

- Machine Learning as a Service
  - JCS'22
  - TrustCom'20 [youtube]
  - ProvSec'20 [youtube]
  - ISPEC'19

- **Federated Learning**
  - **JAIR'22**
  - **ICDE'22 demo [youtube]**
  - **TDSC'22**
  - **IJCAI'23**